Lesson 4 Teacher's Script

Programming Basics III: Direct Hit

VIDEO GAME CREATION

[0:00-0:05] Welcome Back

"Welcome back! Last week you got your character shooting - a key press spawned a bullet that flew up the screen and cleaned itself up. Quick show of hands - who got that working? Anyone try one of the Level 3 ideas - a sound, a distinct bullet look?" [Pause, call on 1-2 volunteers]

"Today your bullets finally have something to hit. By the end of today, your game will have one enemy on screen, and shooting it will destroy it. This is also going to teach us something every single game needs: how does an engine even know two things touched?"

[0:05-0:25] Collision Detection: How Engines Know Things Touched

"Here's the question: your bullet is one object, the enemy is another. As far as the computer's concerned, they're just two separate things with their own positions, updating independently. So how does the game know the exact moment they touch?" [Pause for guesses]

"Every engine's answer is basically the same shape: you give each object a collision shape - usually a simple box or circle, not the exact pixel outline of the artwork - and the engine checks, every frame, whether any two of those shapes overlap. When they do, it fires off a signal or event you can react to. That's collision detection, or sometimes 'overlap detection' when nothing's meant to physically block anything, just detect contact."

[Demo cue: switch to Godot, open a project with bullet.tscn and enemy.tscn already built]

"In Godot, that shape lives on an Area2D node with a CollisionShape2D inside it, and the signal is called area_entered. GameMaker just gives you a Collision event, no shape node needed - you pick the object, and it fires when they overlap. Unity needs a Collider2D on each object, marked 'Is Trigger,' and calls OnTriggerEnter2D. Unreal calls its version Event ActorBeginOverlap. Different names, same underlying question: 'did these two things just start touching?'"

[Live demo] "Let's run this and shoot the enemy." [Fire at the enemy, point at both disappearing] "See how both the bullet and the enemy vanished the instant they overlapped? That's the collision signal firing, and our code reacting to it."

[Discussion question] "Where else in a game do you think this same idea shows up? " (Examples to draw out if needed: picking up a coin, walking through a doorway, an attack landing on an enemy, a character falling into a hazard.)

[Timing checkpoint: wrap by 0:25]

[0:25-0:45] Cleanup Revisited: Destroying Two Things at Once, Safely

"Last week you learned every spawned object needs an explicit cleanup call. Today's twist: one collision needs to clean up TWO objects at once - the bullet that hit, and the enemy that got hit. That raises a real question: what if that cleanup code somehow runs twice for the same enemy? Does the game crash?" [Pause for guesses]

"Good news: in every engine we use, calling 'destroy this' a second time on something that's already been destroyed is completely harmless - the engine just ignores the repeat. That matters, because in a fast-moving game, it's entirely possible for two things to overlap on the exact same frame, and you don't want that rare case to crash the game."

[Demo cue: same Godot project] "Notice something else about where this code lives: it's the Enemy's own script that reacts to being hit - not the Bullet's. The bullet doesn't need to know an enemy exists at all; it just needed a collision shape added to it so the enemy could detect it. Whoever gets hit is the one responsible for deciding what happens next. That's the same 'keep behavior with the object it belongs to' principle from Lesson 3, applied to a new situation."

[Live demo] "Watch what happens if I fire several bullets quickly, including a few that miss." [Fire a rapid burst, some hitting, some missing] "The ones that hit destroy the enemy exactly once. The ones that miss just keep flying and disappear off-screen, exactly like last week - nothing about that changed."

[Discussion question] "Can anyone think of a game moment where you've seen two things need to react to hitting each other at the same time? " (Examples: two characters colliding head-on, a bullet and a shield meeting.)

[Timing checkpoint: wrap by 0:43-0:45]

[0:45-0:50] Placement: Things You Spawn vs. Things You Place

"One more quick distinction before lab. Last week's bullet is spawned while the game is running - it doesn't exist until you press fire. Today's enemy is different: it's placed once, ahead of time, sitting in the scene/room/level from the very start. Both are still instances - copies of a template - they just come into existence at different moments: one at design time, one at runtime."

"You'll see this same split all semester. Some things - bullets, effects, pickups - get created while the game runs. Other things - level layout, a boss, a fixed obstacle - get placed by the designer ahead of time. Today you're doing the second kind for the first time."

[Timing checkpoint: wrap by 0:50]

[0:50-1:00] Today's Exercise & Handouts

"For lab today, open your own project from last week - not a new one. Everything today gets added on top of the character and bullet you already built."

"The goal: one enemy sits on screen, and shooting it destroys both the enemy and the bullet that hit it. Level 1 is getting that working once. Level 2 is making sure it stays clean even with several shots in a row, including ones that miss. Level 3 is polish - a hit sound, a distinct enemy look, or tidying up how the hit-response code is organized."

[Hand out engine-specific handouts]

"Heads up on the most likely snag today: this is the first week that needs a real collision/physics component set up correctly - a Rigidbody2D in Unity, an Area2D with a shape in Godot, overlap

events enabled in Unreal. If nothing happens when you shoot the enemy, that setup is almost always the first thing to check - it's in your handout's 'If Something's Wrong' section."

"Questions before we start lab? ... Let's go make contact."

Lab Time Notes

- The single most common bug this week: "nothing happens when I shoot the enemy." This is almost always a missing or misconfigured collision component - Unity needs a Rigidbody2D on at least one of the two objects; Godot needs a real CollisionShape2D inside each Area2D; Unreal needs "Generate Overlap Events" ticked; 001 Game Creator needs both Actors marked "Solid." Check this first before debugging the logic.
- Second most common: the bullet destroys itself on things it shouldn't, or the enemy never gets destroyed at all - in most engines this traces back to a missing or mismatched tag/group/Scripting ID check.
- Circulate by engine cluster, starting with whichever engine has the most students this week.
- Students who finish Level 2 early should be pointed at the Level 3 ideas on their handout, not given new unrelated work.
- At 1:50, call time and do a 2-minute share-out: ask 2-3 students to show what they built, across different engines if possible.